ShopEZ

E-Commerce Application

Full Stack Development (MERN) — Project Documentation

Team ID	Solo
Team Members	Solo Developer
Ideation Phase	26 / 06 / 2026
Technology Stack	MongoDB, Express.js, React, Node.js (MERN)
Deployment Platform	Render

1. Introduction

Project Title

ShopEZ — Full-Stack E-Commerce Application

Team Members

Team ID: Solo

- Solo Developer — Full Stack Development (Design, Frontend, Backend, Database, Deployment)

Ideation Phase

The project idea was conceptualized and finalized on 26/06/2026.

2. Project Overview

Purpose

ShopEZ is a fully functional, premium full-stack (MERN) e-commerce application built to bridge the gap for small-to-medium retail merchants. It combines an elegant, user-friendly shopping experience for customers with a robust back-end Seller Dashboard for business management and analytics — enabling merchants to manage inventory, track orders, and monitor performance without needing enterprise-level tools.

Features

Customer-Facing Features:

- Homepage with a premium header and fully responsive layout
- Product catalog with live search and category filters (URL-parameter synced, with query encoding for special characters such as "Home & Kitchen")
- Product details page with multi-tab views for specifications and customer reviews
- Shopping cart with a live price calculator that syncs automatically to the database
- 3-step checkout flow supporting Card, UPI, and Cash on Delivery payment methods
- User profile management — editing addresses and viewing order status history

Seller Dashboard Features:

- Analytics panel with live KPI cards for Total Revenue, Total Orders, Pending Orders, and Delivered Orders
- Inventory manager with full CRUD operations (Add, Edit, Delete products via form modals)
- Order manager to view transaction history, search by Order ID, and update shipment status (Pending → Processing → Shipped → Delivered → Cancelled)

3. Architecture

Frontend

The frontend is built with React (using Vite as the build tool) for fast development and optimized production builds. Client-side routing is handled with React Router DOM (v6). React Icons is used for iconography, and React Hot Toast provides non-intrusive notification/alert messages throughout the app. The frontend

communicates with the backend via Axios, using request interceptors to automatically attach JWT tokens to authenticated requests.

Backend

The backend is built on Node.js and Express.js, configured using ES Modules (ESM) syntax. It exposes a set of RESTful API endpoints that handle authentication, product management, cart operations, checkout, and order processing for both customers and sellers.

Database

Data is stored in MongoDB Atlas (cloud-hosted) and accessed through the Mongoose ODM, which defines schemas for Users, Products, Orders, and Carts, and manages the relationships and validations between them.

4. Setup Instructions

Prerequisites

- Node.js (v20.15.0 or later recommended, matching the Render deployment configuration)
- npm (comes bundled with Node.js)
- MongoDB Atlas account (or local MongoDB instance) for the database connection string
- Git, for cloning the repository

Installation

- Clone the repository: **git clone <repository-url>**
- Navigate into the project folder: **cd Project**
- Install backend dependencies: **cd server && npm install**
- Install frontend dependencies: **cd ../client && npm install**
- Create a **.env** file inside the **server** directory with the required environment variables (MongoDB URI, JWT secret, port)
- Ensure **.gitignore** is present so the **.env** file is never committed to GitHub

5. Folder Structure

The project is hosted in a single repository with the following top-level structure:

- **Documentation/** — Project planning documents, requirement sheets, templates, and PDF deliverables
- **Project/** — The complete application codebase
- **.gitignore** — Blocks environment secrets (.env) from being uploaded to GitHub

Client (React Frontend)

The client/ folder contains the React frontend, compiled into static assets for production. It follows a standard Vite + React structure with components, pages, routes, and utility/service files (including the Axios instance configured with request interceptors for JWT attachment).

Server (Node.js Backend)

The server/ folder contains the Express.js backend, organized using ES Modules. It is structured into route handlers, controllers, Mongoose models (User, Product, Order, Cart), authentication middleware, and a seeding script for populating initial data.

6. Running the Application

Frontend:

Run npm run dev (Vite) inside the client directory to start the development server.

Backend:

Run npm start inside the server directory to start the Express API server.

In production, the application is deployed on Render using a unified blueprint deployment (render.yaml) running on Node 20.15.0.

7. API Documentation

The backend exposes RESTful endpoints grouped by resource. Below is a representative summary of the core API groups implemented in ShopEZ:

Endpoint Group	Method(s)	Description
/api/auth	POST	User registration and login; issues a JWT on successful authentication
/api/users/profile	GET, PUT	Fetch or update the logged-in user's profile and address details
/api/products	GET, POST	List all products (with search/category filters) or add a new product (seller only)
/api/products/:id	GET, PUT, DELETE	Fetch, update, or delete a single product (seller only for write operations)
/api/cart	GET, POST, PUT	Fetch, add to, or update the current user's cart, synced with the database
/api/orders	GET, POST	Place a new order (checkout) or fetch the logged-in user's order history
/api/orders/:id/status	PUT	Update the shipment status of an order (seller only)
/api/dashboard/analytics	GET	Fetch KPI data for the Seller Dashboard: revenue, total/pending/delivered orders

Example — Login request/response:

Request: POST /api/auth/login — Body: { "email": "user@example.com", "password": "••••••••" }

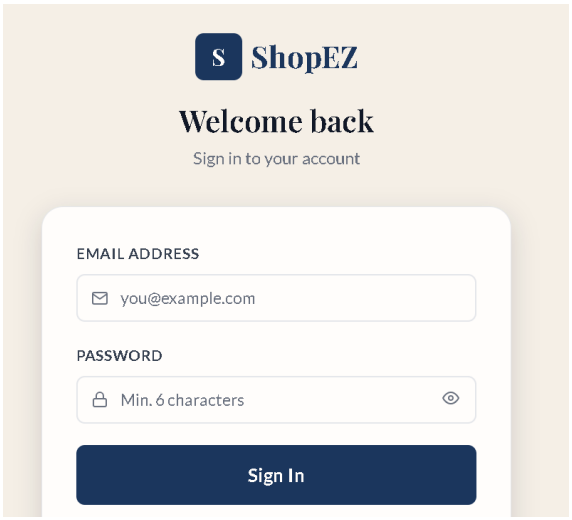
Response: { "token": "<JWT>", "user": { "id": "...", "name": "...", "role": "customer" } }

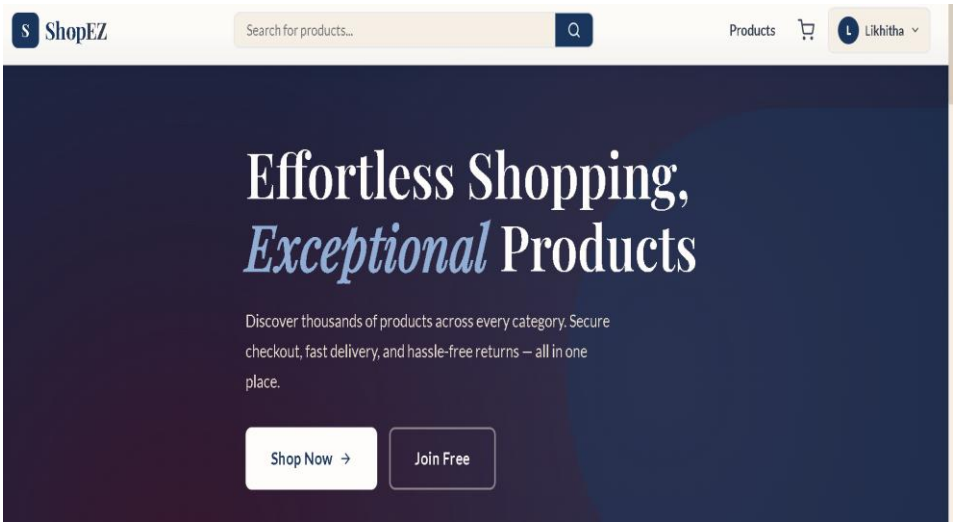
8. Authentication

ShopEZ uses stateless JWT (JSON Web Token) authentication. Upon successful login, the server issues a signed JWT, which is stored in the browser's local storage on the client side. An Axios request interceptor automatically attaches this token as an Authorization header to all outgoing API requests that require authentication. On the backend, middleware verifies the token before granting access to protected routes, and role-based checks (customer vs. seller) restrict access to seller-only operations such as inventory management and order status updates.

9. User Interface

ShopEZ follows a premium "Virtual Closet / Art Gallery" boutique aesthetic rather than a generic e-commerce grid template. The visual design uses a custom, non-generic color palette consisting of Deep Navy Blue, Coral (for vibrant accents and buttons), Warm Ivory (for surfaces and page backgrounds), Light Gray (for borders and dividers), and Charcoal Gray (for primary typography).





10. Testing

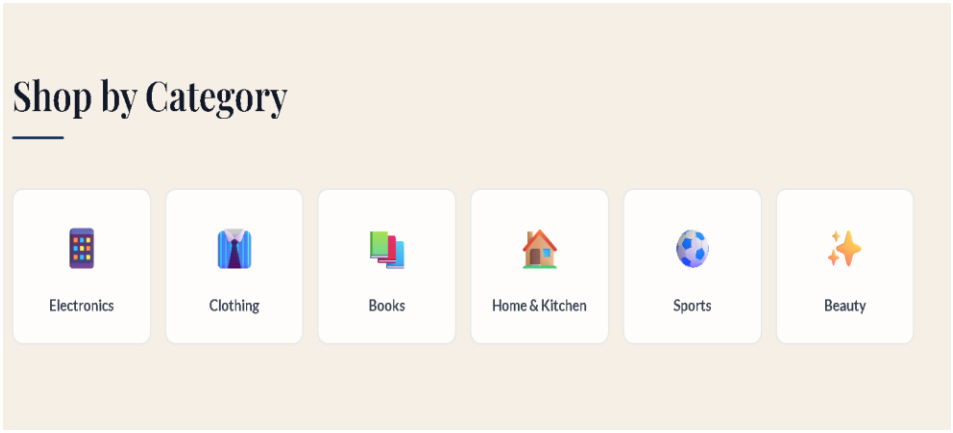
Core functionality — including authentication, cart synchronization, checkout flow, and seller CRUD operations — was verified through manual end-to-end testing across customer and seller user flows. Key

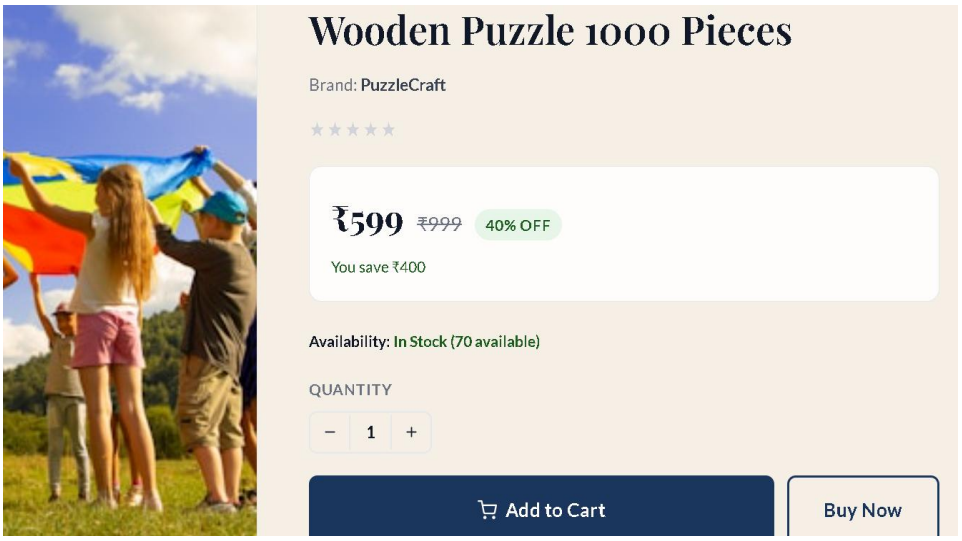
fixes were identified and resolved during this process (see Section 12: Known Issues for details of issues found and corrected).

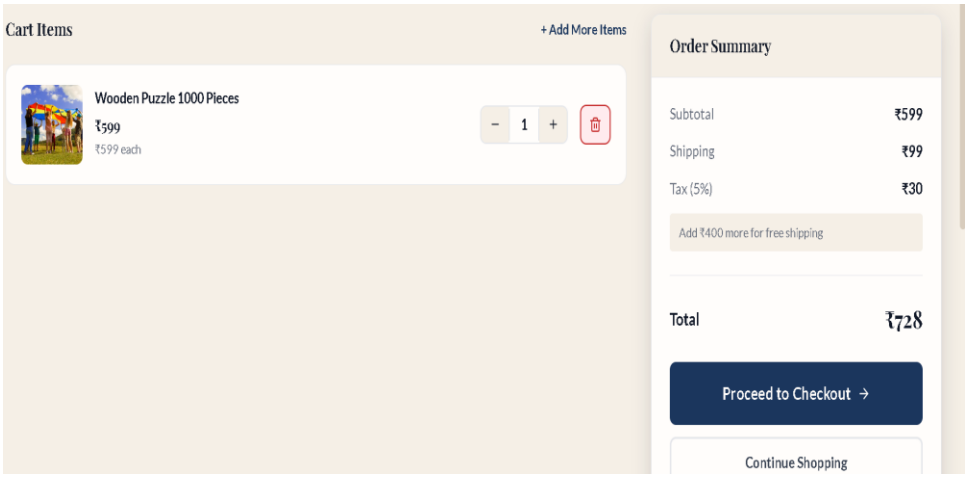
Automated Testing & Verification:

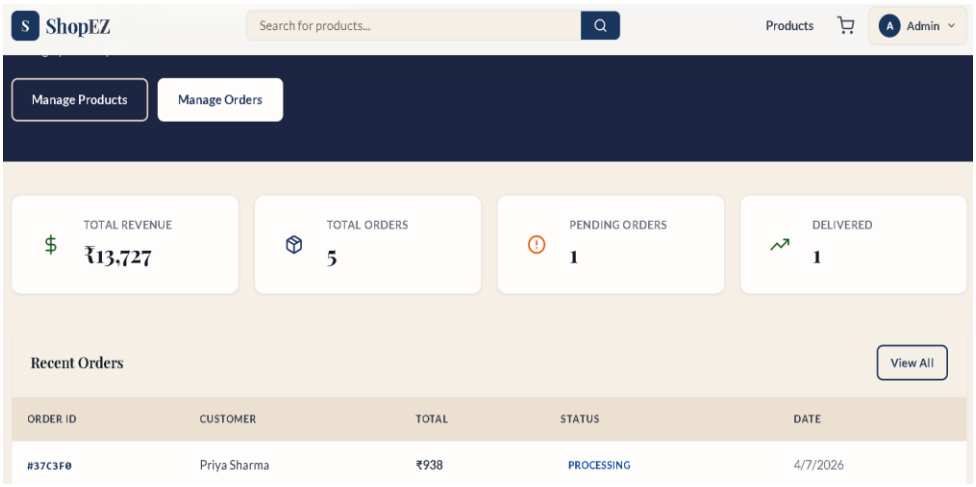
- **Postman API Collections:** Used to test and validate all backend REST endpoints (Authentication, Cart syncing, Order status modifications) and confirm HTTP response status codes.
- **Jest & Supertest:** Implemented backend integration testing to mock HTTP request pipelines and verify database operations under the JWT middleware protection.
- **React Testing Library:** Used to test key UI components (such as ProductCard and ProtectedRoute) in isolation to ensure routing and currency rendering work correctly.

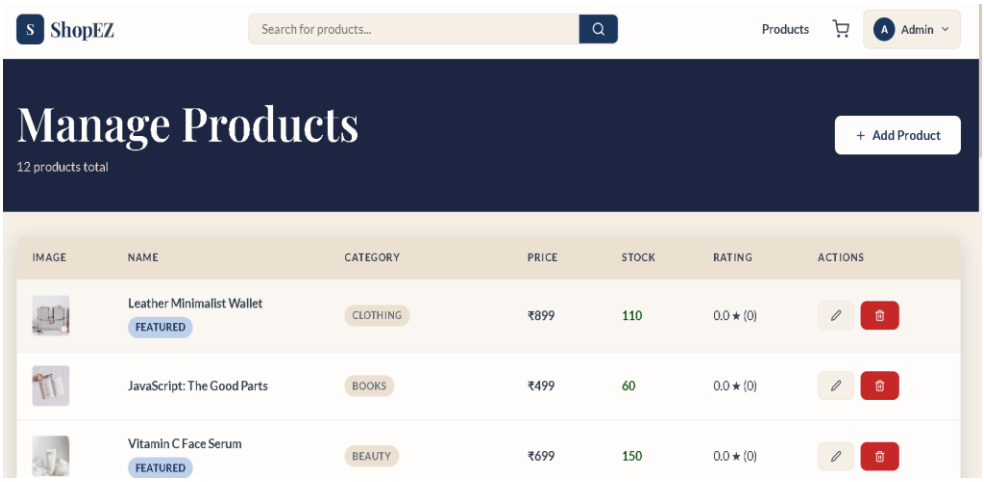
11. Screenshots or Demo











Live demo:

https://shopez-e-commerce-bfxg.onrender.com/

12. Known Issues

The following issues were identified and resolved during development:

- **Database Seeding Bug:** The seeder originally used insertMany, which bypassed Mongoose's pre-save hooks and caused passwords to be stored without proper bcrypt hashing. This was resolved by switching to User.create, ensuring bcrypt password hashes execute correctly.
- **Git Security:** Environment credentials were accidentally exposed in the repository history. These were purged from Git history to restore a clean, secure repository status.
- **Vite Production Build on Render:** Render's production dependency installation excluded devDependencies by default, which broke the Vite build step. This was fixed by using the --include=dev flag during npm install in the deployment configuration.

13. Future Enhancements

- Integration of a real payment gateway (e.g., Razorpay/Stripe) for live Card and UPI transactions
- Email/SMS notifications for order confirmation and shipment status updates
- Product recommendation system based on browsing and purchase history
- Wishlist functionality for customers
- Advanced analytics and sales forecasting for the Seller Dashboard
- Multi-seller marketplace support with individual seller storefronts
- Automated test coverage (unit and integration tests) for both client and server